



Universidade: Universidade Estácio de Sá
Curso: Desenvolvimento Full Stack
Semestre Letivo: 2026.1

Campus: Polo Barra Word - RJ
Disciplina: Programação Back-end Com Java

Aluno: Claudia Ariane Duarte da Silva
MAT: 202408470801

Repositório Git:
<https://github.com/Claudiaduart/Trabalho-DGT2821.git>

Relatório de Acompanhamento de Prática 1º Procedimento

1. Título da Prática

Criação das Entidades e Sistema de Persistência

2. Objetivo da Prática

Aplicar conceitos de Programação Orientada a Objetos (POO) em Java, como herança e polimorfismo, e implementar a persistência de objetos em disco utilizando arquivos binários e a interface `Serializable`. O projeto visa também aplicar padrões de projeto para o gerenciamento de dados através de repositórios.

3. Códigos Solicitados

- Entidades (Pacote `model`):

```
0 Pessoa.java

package model;

import java.io.Serializable;

public class Pessoa implements Serializable {
    private int id;
    private String nome;

    public Pessoa() {
```

```

    }

    public Pessoa(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public void exhibir() {
        System.out.println("ID: " + id);
        System.out.println("Nome: " + nome);
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}

```

0 PessoaFisica.java

```

package model;

import java.io.Serializable;

public class PessoaFisica extends Pessoa implements Serializable {
    private String cpf;
    private int idade;

    public PessoaFisica() {
        super();
    }

    public PessoaFisica(int id, String nome, String cpf, int idade) {
        super(id, nome);
        this.cpf = cpf;
        this.idade = idade;
    }

    @Override
    public void exhibir() {
        super.exibir();
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }

    public String getCpf() {
        return cpf;
    }
}

```

```

    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }
}

```

0 PessoaJuridica.java

```

package model;

import java.io.Serializable;

public class PessoaJuridica extends Pessoa implements Serializable {
    private String cnpj;

    public PessoaJuridica() {
        super();
    }

    public PessoaJuridica(int id, String nome, String cnpj) {
        super(id, nome);
        this.cnpj = cnpj;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CNPJ: " + cnpj);
    }

    public String getCnpj() {
        return cnpj;
    }

    public void setCnpj(String cnpj) {
        this.cnpj = cnpj;
    }
}

```

• Gerenciadores / Repositórios (Pacote model):

0 PessoaFisicaRepo.java

```

package model;

import java.io.*;
import java.util.ArrayList;

```

```

public class PessoaFisicaRepo {
    private ArrayList<PessoaFisica> listaPessoaFisica = new ArrayList<>();

    public void inserir(PessoaFisica pf) {
        listaPessoaFisica.add(pf);
    }

    public void alterar(PessoaFisica pf) {
        for (int i = 0; i < listaPessoaFisica.size(); i++) {
            if (listaPessoaFisica.get(i).getId() == pf.getId()) {
                listaPessoaFisica.set(i, pf);
                break;
            }
        }
    }

    public void excluir(int id) {
        listaPessoaFisica.removeIf(pf -> pf.getId() == id);
    }

    public PessoaFisica obter(int id) {
        for (PessoaFisica pf : listaPessoaFisica) {
            if (pf.getId() == id) {
                return pf;
            }
        }
        return null;
    }

    public ArrayList<PessoaFisica> obterTodos() {
        return listaPessoaFisica;
    }

    public void persistir(String arquivo) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(arquivo))) {
            oos.writeObject(listaPessoaFisica);
        }
    }

    @SuppressWarnings("unchecked")
    public void recuperar(String arquivo) throws IOException, ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(arquivo))) {
            listaPessoaFisica = (ArrayList<PessoaFisica>) ois.readObject();
        }
    }
}

```

0 PessoaJuridicaRepo.java

```

package model;

import java.io.*;
import java.util.ArrayList;

public class PessoaJuridicaRepo {
    private ArrayList<PessoaJuridica> listaPessoaJuridica = new ArrayList<>();

    public void inserir(PessoaJuridica pj) {

```

```

        listaPessoaJuridica.add(pj);
    }

    public void alterar(PessoaJuridica pj) {
        for (int i = 0; i < listaPessoaJuridica.size(); i++) {
            if (listaPessoaJuridica.get(i).getId() == pj.getId()) {
                listaPessoaJuridica.set(i, pj);
                break;
            }
        }
    }

    public void excluir(int id) {
        listaPessoaJuridica.removeIf(pj -> pj.getId() == id);
    }

    public PessoaJuridica obter(int id) {
        for (PessoaJuridica pj : listaPessoaJuridica) {
            if (pj.getId() == id) {
                return pj;
            }
        }
        return null;
    }

    public ArrayList<PessoaJuridica> obterTodos() {
        return listaPessoaJuridica;
    }

    public void persistir(String arquivo) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(arquivo))) {
            oos.writeObject(listaPessoaJuridica);
        }
    }

    @SuppressWarnings("unchecked")
    public void recuperar(String arquivo) throws IOException,
ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(arquivo))) {
            listaPessoaJuridica = (ArrayList<PessoaJuridica>) ois.readObject();
        }
    }
}

```

- **Classe Principal:**

```

0  CadastroPOO.java

```

```

package cadastropoo;

```

```

import model.PessoaFisica;
import model.PessoaFisicaRepo;
import model.PessoaJuridica;
import model.PessoaJuridicaRepo;

public class CadastroPOO {

    public static void main(String[] args) {
        try {
            // ===== PESSOA FÍSICA =====
            PessoaFisicaRepo repo1 = new PessoaFisicaRepo();

            repo1.inserir(new PessoaFisica(1, "João Silva", "11122233344", 30));
            repo1.inserir(new PessoaFisica(2, "Maria Souza", "55566677788", 25));

            String arquivoFisica = "pessoas_fisicas.bin";
            repo1.persistir(arquivoFisica);

            PessoaFisicaRepo repo2 = new PessoaFisicaRepo();

            repo2.recuperar(arquivoFisica);

            System.out.println("Dados de Pessoa Física Recuperados:");
            for (PessoaFisica pf : repo2.obterTodos()) {
                pf.exibir();
                System.out.println(); // Quebra de linha para organização
            }

            // ===== PESSOA JURÍDICA =====
            PessoaJuridicaRepo repo3 = new PessoaJuridicaRepo();

            repo3.inserir(new PessoaJuridica(3, "Tech Solutions Ltda", "12345678000199"));
            repo3.inserir(new PessoaJuridica(4, "Comercial Alpha S.A.", "98765432000111"));

            String arquivoJuridica = "pessoas_juridicas.bin";
            repo3.persistir(arquivoJuridica);

            PessoaJuridicaRepo repo4 = new PessoaJuridicaRepo();

            repo4.recuperar(arquivoJuridica);

            System.out.println("Dados de Pessoa Jurídica Recuperados:");
            for (PessoaJuridica pj : repo4.obterTodos()) {
                pj.exibir();
                System.out.println(); // Quebra de linha para organização
            }

        } catch (Exception e) {
            System.out.println("Ocorreu um erro: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

4. Resultados da Execução

Ao executar o método `main` da classe `CadastroPOO`, o sistema cria instâncias de objetos, persiste esses dados nos arquivos `pessoas_fisicas.bin` e `pessoas_juridicas.bin`, os recupera em novos repositórios e exibe o resultado no console.

Saída no Console (Output):

Dados de Pessoa Física Recuperados:

ID: 1

Nome: João Silva

CPF: 11122233344

Idade: 30

ID: 2

Nome: Maria Souza

CPF: 55566677788

Idade: 25

Dados de Pessoa Jurídica Recuperados:

ID: 3

Nome: Tech Solutions Ltda

CNPJ: 12345678000199

ID: 4

Nome: Comercial Alpha S.A.

CNPJ: 98765432000111

5. Análise e Conclusão

Quais as vantagens e desvantagens do uso de herança?

- **Vantagens:** Reaproveitamento de código e organização. Não precisamos digitar `id` e `nome` várias vezes, pois a classe `Pessoa` já fornece isso para as outras.
- **Desvantagens:** Cria uma forte dependência. Se mudarmos algo na classe principal (`Pessoa`), todas as classes filhas podem dar erro. Além disso, no Java, uma classe não pode ter mais de uma "mãe" (herança múltipla não é permitida).

Por que a interface `Serializable` é necessária ao efetuar persistência em arquivos binários? Ela funciona como um "selo de autorização". Ela avisa ao Java que o nosso objeto é seguro e tem permissão para ser transformado em bytes (dados binários) para ser gravado e lido no disco sem causar erros de segurança.

Como o paradigma funcional é utilizado pela API `stream` no Java? Ele permite escrever códigos mais curtos e diretos, dizendo "o que" fazer em vez de escrever todo o passo a passo de "como" fazer. No projeto, usamos isso de forma prática com funções curtas (lambdas), como no comando `removeIf`, que acha e apaga um item da lista em uma única linha, sem precisar de grandes blocos de `for` e `if`.

Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos? Usamos o padrão **Repository** (Repositório) ou **DAO** (*Data Access Object*). O objetivo desse padrão é separar totalmente a parte do código que mexe com os dados (arquivos) da parte que interage com o usuário. Assim, fica muito mais fácil dar manutenção no sistema.

Relatório de Acompanhamento de Prática 2º

Procedimento

1. Título da Prática

Criação do Cadastro em Modo Texto

2. Objetivo da Prática

Implementar uma interface de usuário em modo texto (CLI - *Command Line Interface*) para o sistema de cadastro utilizando a classe `Scanner`. O objetivo é permitir a interação dinâmica com o usuário, viabilizando operações de inclusão, alteração, exclusão, busca e listagem de entidades (Pessoa Física e Jurídica), além de integrar essas funcionalidades aos repositórios e ao sistema de persistência em arquivos binários criados no procedimento anterior.

3. Códigos Solicitados

Classe CadastroPOO.java (Pacote cadastrapoo)

Java

```
package cadastrapoo;
```

```
import java.util.Scanner;
import model.PessoaFisica;
import model.PessoaFisicaRepo;
import model.PessoaJuridica;
import model.PessoaJuridicaRepo;
```

```
public class CadastroPOO {
```

```
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        PessoaFisicaRepo repoFisica = new PessoaFisicaRepo();
        PessoaJuridicaRepo repoJuridica = new PessoaJuridicaRepo();
        int opcao;

        do {
            System.out.println("=====");
            System.out.println("1 - Incluir Pessoa");
            System.out.println("2 - Alterar Pessoa");
            System.out.println("3 - Excluir Pessoa");
            System.out.println("4 - Buscar pelo Id");
            System.out.println("5 - Exibir Todos");
            System.out.println("6 - Persistir Dados");
```



```

System.out.println("7 - Recuperar Dados");
System.out.println("0 - Finalizar Programa");
System.out.println("=====");

opcao = Integer.parseInt(scanner.nextLine());

if (opcao == 0) {
    break;
}

if (opcao >= 1 && opcao <= 5) {
    System.out.println("F - Pessoa Fisica | J - Pessoa Juridica");
    String tipo = scanner.nextLine().toUpperCase();

    if (opcao == 1) {
        System.out.println("Digite o id da pessoa:");
        int id = Integer.parseInt(scanner.nextLine());

        System.out.println("Insira os dados...");
        System.out.print("Nome: ");
        String nome = scanner.nextLine();

        if (tipo.equals("F")) {
            System.out.print("CPF: ");
            String cpf = scanner.nextLine();
            System.out.print("Idade: ");
            int idade = Integer.parseInt(scanner.nextLine());
            repoFisica.inserir(new PessoaFisica(id, nome, cpf, idade));
        } else if (tipo.equals("J")) {
            System.out.print("CNPJ: ");
            String cnpj = scanner.nextLine();
            repoJuridica.inserir(new PessoaJuridica(id, nome, cnpj));
        }
    }
    else if (opcao == 2) {
        System.out.println("Digite o id da pessoa:");
        int id = Integer.parseInt(scanner.nextLine());

        if (tipo.equals("F")) {
            PessoaFisica pf = repoFisica.obter(id);
            if (pf != null) {
                System.out.println("Insira os novos dados...");
                System.out.print("Nome: "); String nome = scanner.nextLine();
                System.out.print("CPF: "); String cpf = scanner.nextLine();
                System.out.print("Idade: "); int idade = Integer.parseInt(scanner.nextLine());
                repoFisica.alterar(new PessoaFisica(id, nome, cpf, idade));
            }
        } else if (tipo.equals("J")) {
            PessoaJuridica pj = repoJuridica.obter(id);
            if (pj != null) {
                System.out.println("Insira os novos dados...");
                System.out.print("Nome: "); String nome = scanner.nextLine();
            }
        }
    }
}

```

```

        System.out.print("CNPJ: "); String cnpj = scanner.nextLine();
        repoJuridica.alterar(new PessoaJuridica(id, nome, cnpj));
    }
}
}
else if (opcao == 3) {
    System.out.println("Digite o id da pessoa:");
    int id = Integer.parseInt(scanner.nextLine());

    if (tipo.equals("F")) repoFisica.excluir(id);
    else if (tipo.equals("J")) repoJuridica.excluir(id);
}
else if (opcao == 4) {
    System.out.println("Digite o id da pessoa:");
    int id = Integer.parseInt(scanner.nextLine());

    if (tipo.equals("F")) {
        PessoaFisica pf = repoFisica.obter(id);
        if (pf != null) pf.exibir();
    } else if (tipo.equals("J")) {
        PessoaJuridica pj = repoJuridica.obter(id);
        if (pj != null) pj.exibir();
    }
}
else if (opcao == 5) {
    if (tipo.equals("F")) {
        for (PessoaFisica pf : repoFisica.obterTodos()) pf.exibir();
    } else if (tipo.equals("J")) {
        for (PessoaJuridica pj : repoJuridica.obterTodos()) pj.exibir();
    }
}
}
else if (opcao == 6 || opcao == 7) {
    System.out.print("Prefixo dos arquivos: ");
    String prefixo = scanner.nextLine();

    try {
        if (opcao == 6) {
            repoFisica.persistir(prefixo + ".fisica.bin");
            repoJuridica.persistir(prefixo + ".juridica.bin");
            System.out.println("Dados salvos com sucesso.");
        } else {
            repoFisica.recuperar(prefixo + ".fisica.bin");
            repoJuridica.recuperar(prefixo + ".juridica.bin");
            System.out.println("Dados recuperados com sucesso.");
        }
    } catch (Exception e) {
        System.out.println("Erro: " + e.getMessage());
    }
}
} while (opcao != 0);

```

```
        scanner.close();
    }
}
```

4. Resultados da Execução

Abaixo está a representação da saída do console durante a execução do programa, demonstrando a interação para inclusão de uma nova Pessoa Física:

Saída no Console:

```
=====
1 - Incluir Pessoa
2 - Alterar Pessoa
3 - Excluir Pessoa
4 - Buscar pelo Id
5 - Exibir Todos
6 - Persistir Dados
7 - Recuperar Dados
0 - Finalizar Programa
=====
1
F - Pessoa Fisica | J - Pessoa Juridica
F
Digite o id da pessoa:
120
Insira os dados...
Nome: Joao
CPF: 11122233344
Idade: 30
```

5. Análise e Conclusão

O que são elementos estáticos e qual o motivo para o método `main` adotar esse modificador? Elementos estáticos (variáveis ou métodos) são aqueles que pertencem à classe em si, e não a um objeto criado a partir dela (não precisam do comando `new` para existir). O método `main` precisa ser estático porque ele é o "botão de ligar" do programa; o Java precisa conseguir rodá-lo imediatamente, antes mesmo de criar qualquer objeto na memória.

Para que serve a classe `Scanner`? No nosso projeto, ela serve para capturar tudo o que o usuário digita no teclado durante o programa. Ela pega o texto cru da tela preta e facilita a conversão disso para formatos que o Java entende, como textos (`String`) ou números (`int`).

Como o uso de classes de repositório impactou na organização do código? Deixou o projeto muito mais limpo. O código ficou dividido por responsabilidades: a classe principal (`CadastroPOO`) cuidou apenas de desenhar o menu e conversar com o usuário, enquanto as classes de repositório focaram apenas no "trabalho pesado" de salvar, buscar e excluir dados.

